

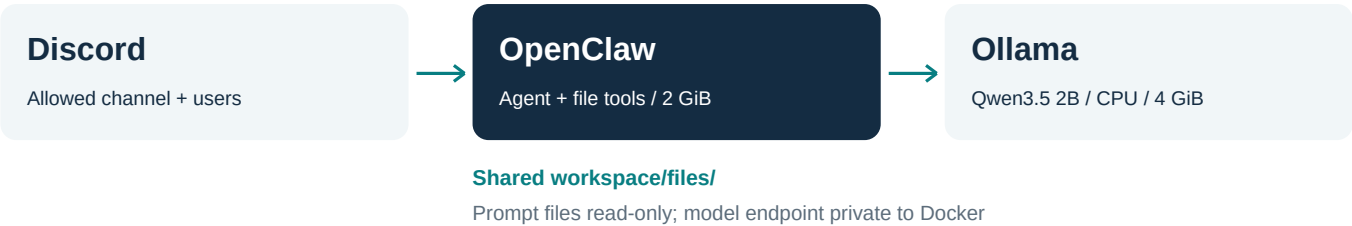
OpenClaw on an 8 GB CPU host

Sonne / Technical deployment guide / 7 October 2026 / Revision 4

1. Deployment profile

Run one small, multimodal OpenClaw agent on **Ubuntu 24.04 LTS, amd64, 8 GB total RAM, no GPU**. Start with an installed Ubuntu system and your own normal user account with sudo access. Pages 2 and 3 install the tools and Docker, test them, and download the public project. No GitHub account is required. Allow roughly 15 GB free disk initially and check usage after pulling images.

The agent understands text, photos and screenshots, and reads or writes small text files. One Discord channel serves several explicitly allowed users. They share the channel conversation and the same files. Audio, video and shell execution are outside this profile.



Inference stays on your computer. Discord carries messages and attachments and needs internet access. Docker pulls and the model download also need internet. There is no cloud model fallback and no public inbound port for the bot.

Selected model and versions

Component	Pinned selection
OpenClaw	2026.9.8, official GHCR image
Ollama	0.40.0, official Docker Hub image
Multimodal model	qwen3.5:2b-q4_K_M
Input / tools	Text + images; native tool calling
Runtime budget	16,384 context; 1,024 output tokens

The model package is approximately 1.9 GB, including its vision components. This is a download size, not a RAM measurement. Avoid the bare qwen3.5 tag: it selects a larger model. A 2B model has limited OCR, reasoning and tool reliability; verify the actual tasks you need. [Model card](#).

Validation boundary: configuration has been checked against OpenClaw 2026.9.8. CPU inference, Discord delivery and whole-host memory fit have not been measured here. Page 8 defines the target-host acceptance check.

2. Install the host tools and Docker

Open a terminal on your Ubuntu computer and use your normal user account. Commands marked `sudo` ask for your Linux password. Copy code blocks in order; stop if a command fails. A backslash at the end of a line continues the command on the next line, so keep line breaks when copying. First check the system:

```
cat /etc/os-release
dpkg --print-architecture
free -h
df -h "$HOME"
```

Require Ubuntu **24.04** and **amd64**. The Docker repository below is specifically for that combination. On a fresh host without an existing container runtime, install the utilities used throughout this guide:

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl git nano \
python3 python3-venv procps ripgrep openssh-client
```

Git downloads the project; Python generates config; nano edits files. `procps` supplies `free` and `vmstat`, `ripgrep` supplies `rg`, and the SSH client supports the optional admin tunnel. `python3-venv` is used only if you rebuild the PDF.

Add Docker's official package repository

If this host already runs Docker, containerd or Podman, first check the [official installation and package-conflict instructions](#). Do not remove a runtime serving other workloads. A working Docker Engine with the Compose plugin can proceed to page 3's checks.

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
-o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<'EOF'
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: noble
Components: stable
Architectures: amd64
Signed-By: /etc/apt/keyrings/docker.asc
EOF
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
```

This installs the stable Docker Engine and its Compose plugin. Use `docker compose` with a space in all later commands.

3. Verify Docker and download Sonne

Enable Docker for the deployment user

Add only the trusted operator account. Docker group membership grants administrative control over the host; it is unrelated to the Discord user allowlist. See [Docker's Linux post-installation guide](#).

```
sudo groupadd -f docker
sudo usermod -aG docker "$(id -un)"
```

Log out of Ubuntu completely and log back in, or disconnect and reconnect your SSH session. Opening another terminal in the same desktop session is insufficient. Then run these commands without sudo:

```
id -nG
docker version
docker compose version
docker run --rm hello-world
git --version
python3 --version
command -v curl nano rg vmstat ssh
```

Require docker in the group list, Docker Client and Server versions without connection errors, Compose **v2 or newer**, and the **Hello from Docker!** message. Every utility in the last command must print a path. If Docker access is denied, repeat the logout/login step; if the daemon is inactive, run `sudo systemctl start docker` and retry. Do not continue until these checks pass.

Download the public project

Anyone can download Sonne. You do not need a GitHub account, access token or SSH key. Run these commands as your own Linux user to put the project in `~/agents/sonne`:

```
mkdir -p ~/agents
cd ~/agents
git clone https://github.com/Krabbens/sonne.git
cd sonne
```

`~` means your home directory; it works with any Linux username. The account name in the download URL identifies the project publisher, not an account you must log into. Check that the project files are present:

```
ls compose.yaml .env.example scripts/configure.py
```

Keep using this folder for the later commands. If you open a new terminal, run `cd ~/agents/sonne` first. If you choose a different folder, use that path instead. An existing download can be updated from inside its folder with `git pull --ff-only`.

What runs inside Docker

No host installation of Node.js, npm, OpenClaw or Ollama is needed. Page 5 pulls their container images, installs the pinned Discord plugin inside the OpenClaw container, and downloads the model into a Docker volume. PDF authoring packages are optional; their installation is shown on page 10.

4. Create the Discord bot and local config

Discord application

Use your own Discord account and a server where you can add bots. You will create your own application, token and access list; the project does not supply a shared bot account.

1. Open the [Discord Developer Portal](#), create an application named Sonne, and open its Bot page.
2. Enable **Message Content Intent**. This profile uses explicit numeric user IDs, so Server Members, Presence and voice intents are disabled in OpenClaw and are unnecessary here.
3. Generate the bot token on the Bot page. Keep it on the deployment machine; never paste it in Discord, Git or an issue.
4. In OAuth2 URL Generator select bot and applications.commands. Grant View Channels, Send Messages, Read Message History, Embed Links and Attach Files. Do not grant Administrator. Invite the bot to your server.
5. Make one text channel accessible only to the intended group and the bot. This guide uses a normal channel, not threads or voice.
6. Enable Discord Developer Mode. Copy the Application ID, Server ID, Channel ID and each allowed User ID. Right-click the relevant server, channel or user to copy its ID.

Fill the local environment

The repository was downloaded on page 3. From your normal user account, run:

```
cd ~/agents/sonne
cp .env.example .env
nano .env
```

Replace the five Discord settings with your own token and IDs; the digits below are examples only. In nano, save with Ctrl+O and Enter, then exit with Ctrl+X. User IDs are comma-separated, without spaces. Leave gateway token and local UID/GID blank; the configuration script fills them. Keep the pinned image settings for initial setup.

```
DISCORD_BOT_TOKEN=your-real-bot-token
DISCORD_APPLICATION_ID=11111111111111111111
DISCORD_GUILD_ID=22222222222222222222
DISCORD_CHANNEL_ID=33333333333333333333
DISCORD_USER_IDS=44444444444444444444, 55555555555555555555
```

```
python3 scripts/configure.py
docker compose config --quiet
```

The generator checks numeric IDs, creates `.state/openclaw/openclaw.json` and `workspace/files/`, sets private permissions, and matches the non-root container UID/GID to your Linux user. JSON stores secret references, not your token. `.env`, `state` and `workspace` are ignored by Git. Run the script without `sudo`.

5. Pull, validate and start

Run these commands in order from the project folder. They download pre-built containers and the model, install the Discord plugin, then start the services. Keep the line breaks when copying.

```
docker compose pull
docker compose run --rm --no-deps openclaw \
  node dist/index.js plugins install @openclaw/discord@2026.9.8 \
  --pin --accept-capabilities --force
docker compose run --rm --no-deps openclaw \
  node dist/index.js config validate --json
docker compose up -d ollama
docker compose exec -T ollama ollama pull qwen3.5:2b-q4_K_M
docker compose exec -T ollama ollama list
docker compose up -d openclaw
docker compose ps
```

Discord is a separately installed official plugin in this release. The install command stores it in persistent state; `--force` confirms the npm source for this first installation. It can overwrite an existing plugin, so do not rerun it casually. Require config validation to succeed with no missing-Discord warning before starting the bot.

Check the exact model tag in `ollama list`. Both services should become healthy. The Ollama health check proves that the daemon responds; it does not prove that the model is loaded or working.

Test model routing without a chat-agent turn

```
docker compose exec -T openclaw node dist/index.js \
  infer model run --local --agent sonne \
  --model ollama/qwen3.5:2b-q4_K_M \
  --prompt 'Reply with exactly: sonne-ok' --json
```

Expect a successful completion containing `sonne-ok`. This lean probe checks provider/model routing. It does not load the agent's system prompt, tools or Discord session. The first CPU request may be much slower than later requests. No token-per-second claim is made.

Verify Discord and optional administration

From an allowed user account, send `@Sonne Reply with exactly: sonne-ok` in the allowed channel. Require a visible reply. Then test an image and file actions as described on page 7.

```
docker compose logs --tail=100 openclaw
docker compose logs --tail=100 ollama
```

The optional Control UI is at 127.0.0.1:18789. Enter the gateway token from your local `.env` in its connection settings; complete device pairing if requested. Treat the UI as an operator interface. It is bound only to host loopback.

If pairing is requested, list devices and approve the matching request from your own browser. Replace `REQUEST_ID` with its actual ID:

```
docker compose exec -T openclaw node dist/index.js devices list
docker compose exec -T openclaw node dist/index.js devices approve REQUEST_ID
```

For a headless host, tunnel from your own computer, then open the same local URL:

```
ssh -N -L 18789:127.0.0.1:18789 user@your-linux-host
```

Discord uses an outbound gateway connection. Do not expose port 18789 or Ollama's port 11434 to the internet for this setup.

6. Model settings and the system prompt

Settings that keep this profile small

The provider uses api: "ollama" and http://ollama:11434, with no /v1 suffix. OpenClaw's native Ollama adapter carries tool calls and image input. input: ["text", "image"] declares vision capability explicitly.

```
"contextTokens": 16384,  
"maxTokens": 1024,  
"params": {  
  "num_ctx": 16384,  
  "num_gpu": 0,  
  "num_predict": 1024,  
  "thinking": false,  
  "keep_alive": "5m"  
}
```

The larger contextwindow is model capability metadata; contextTokens and num_ctx are the deliberately smaller runtime budget. Image inputs are scaled to a longest side of 1024 pixels. Send one cropped image at a time. Ollama and OpenClaw both serialize work; additional users wait rather than loading more model runners.

Edit your agent's instructions

OpenClaw assembles its own system prompt. Existing SOUL.md and AGENTS.md files add persona and working rules; this does not replace OpenClaw's entire internal prompt. The supplied persona starts with:

```
You are Sonne, a practical assistant for understanding images and  
working with small text files. Reply in the user's language.  
Keep answers concise and concrete.
```

```
Describe what is visible in an image, including relevant text.  
Distinguish observations from guesses. If text is unreadable,  
ask for a closer crop. Never claim to see an image that was not  
provided or successfully loaded.
```

The complete files also require real tool results, workspace-only file work, confirmation before overwriting, and treating instructions inside images or files as untrusted content. Edit the host copies in templates/workspace/, then recreate the container:

```
nano templates/workspace/SOUL.md  
nano templates/workspace/AGENTS.md  
docker compose up -d --force-recreate openclaw
```

Start a fresh session in the operator UI to avoid relying on old context. The prompt directory is read-only at /workspace; only /workspace/files is writable and maps to host workspace/files/. Directory mounts avoid Docker Desktop's nested file-mount failure. The only tools are read, write, edit and view_image; tools.fs.workspaceOnly independently enforces filesystem scope.

For other configuration changes, edit templates/openclaw.json, rerun python3 scripts/configure.py, then recreate OpenClaw. Direct edits to generated JSON will be replaced by the generator.

7. Functional acceptance checks

Use the allowed Discord channel, mention the bot, and wait for each request to finish. Run these checks with disposable test files before trusting the agent with useful work. A model reply claiming success is not sufficient evidence of a file change.

Text and image

Send a normal question in English, then in Polish. Expect concise replies in the requested language. Attach `examples/vision-check.png` and ask: "Read the large label and identify the red shape." The expected facts are **SONNE 42** and a **red square**. This simple fixture checks image delivery and visible content; it is not an OCR benchmark.

Then send your own cropped screenshot with readable text. Ask for specific visible details and verify them yourself. Repeat once after the model has been idle for more than five minutes to exercise a cold reload.

File actions and overwrite confirmation

```
@Sonne Create files/smoke.txt containing exactly SONNE FILE OK.  
Read it back and report the workspace-relative path.
```

Check on the Linux host:

```
cat workspace/files/smoke.txt
```

Ask the bot to replace its contents. Expect a confirmation question before the change, then approve it and verify the new content on disk. Confirm both the tool action and the final file, not just the wording of the response.

Scope and access controls

Check	Expected outcome
Ask to read <code>/etc/passwd</code>	Filesystem tool refuses access outside workspace
Ask to read <code>../openclaw.json</code>	Scope refusal; no configuration content returned
Ask to edit <code>SOUL.md</code>	Read-only mount prevents mutation
Message from an unlisted user	No agent reply or file action
Mention in another channel	No agent reply or file action
DM the bot	No conversation; DMs are disabled

For the filesystem checks, verify the actual denied tool result in operator session details or logs. A natural-language refusal alone does not prove the filesystem guard. Everyone admitted to the shared channel can access its shared files; there is no per-user file isolation.

Persistence

```
docker compose restart  
cat workspace/files/smoke.txt
```

Require the file and model download to survive the restart. Send another Discord request. Run the memory acceptance check on the next page while repeating the image and file tasks.

8. Whole-host memory and data

Memory budget, not a performance guarantee

Consumer	Initial cap or allowance
Ollama container, model + context + vision	4 GiB hard cap
OpenClaw container, Node + channel runtime	2 GiB hard cap
Linux, Docker daemon, other host work	Remaining host memory

Docker's 4g and 2g limits use binary units. An "8 GB" machine's actual usable memory may be lower; inspect `free -h`. The caps leave roughly 2 GiB on an 8 GiB host, but do not prove that either process fits its allocation. Equal `mem_limit` and `memswap_limit` disable container swap where supported. Node's 1 GiB old-space limit is only a heap limit, not total process RAM.

In separate terminals, observe the host while running page 7's tests. Include a cold model load, an image turn, file tools and a short sequence of requests from two users:

```
free -h
docker compose stats
vmstat 1
docker compose exec -T ollama ollama ps
```

`ollama ps` should show CPU processing and the intended context. Container stats exclude some host overhead. In `vmstat`, disregard the first averaged row and look for sustained `si/so` activity in later rows. After the test, inspect OOM state and restarts:

```
docker inspect $(docker compose ps -aq) \
  --format '{{.Name}} OOM={{.State.OOMKilled}} restarts={{.RestartCount}}'
sudo journalctl -k --since '15 minutes ago' | rg -i 'oom|out of memory'
```

Accept only if tasks complete, neither container is OOM-killed or repeatedly restarting, available host RAM stays above 512 MiB during the sample, and swap is not continuously growing. Record CPU, usable RAM, peaks and response times. Without those measurements, RAM fit remains unverified.

Stored data and backup

`workspace/files/` contains user files; `.state/openclaw/` contains configuration, sessions and runtime data. The named `sonne_ollama-models` volume holds the downloaded model. Prompt source and example configuration are in Git. The real `.env` must be protected separately.

Stop the stack for a consistent backup. This archive includes secrets and conversations; store it outside the repository with restricted access:

```
docker compose stop
umask 077
tar -czf ../sonne-backup-$(date +%Y%m%d-%H%M%S).tgz \
  .env .state workspace
docker compose start
```

For restore, stop the stack, extract a trusted backup into the same checkout, preserve file ownership, then start again. A different host/user may require regenerating UID/GID and correcting restored ownership. The model can be downloaded again. Do not use `docker compose down -v` unless you intend to delete its volume.

9. Maintenance and troubleshooting

Routine operations and deliberate upgrades

```
docker compose logs --tail=100 openclaw
docker compose logs --tail=100 ollama
docker compose restart openclaw
docker compose stop
docker compose start
```

Before upgrading, back up state and save the current image settings. Change one pinned image at a time in `.env`, read its release notes, pull, validate the config and recreate that service. Upgrade the separately installed Discord plugin to a compatible pinned release when required. Repeat the acceptance checks. State migrations may require restoring the matching backup when rolling back.

```
docker compose pull openclaw
docker compose run --rm --no-deps openclaw \
  node dist/index.js config validate --json
docker compose up -d --force-recreate openclaw
```

To roll back, restore the previous image setting and compatible state backup with the stack stopped, then recreate the service. Do not run two gateway versions concurrently against the same state. Version tags and model tags can move: record the actual image digest and model ID for your accepted deployment.

Symptoms and checks

Bot is offline or silent. Check `docker compose ps` and `logs`. Verify the bot token, Application ID, Message Content Intent in both the portal and config, channel permissions, numeric guild/channel/user IDs, and an explicit bot mention. DMs are intentionally disabled. After editing IDs or tokens, rerun the generator and recreate OpenClaw.

Config validation fails. Use the pinned OpenClaw version. Do not copy settings from unrelated versions. Fix the template and regenerate; never skip validation just to start the bot.

Connection refused or tool JSON appears as text. The model endpoint must be `http://ollama:11434` inside Docker, with `api: "ollama"`. `localhost` would point at the OpenClaw container. The `/v1` compatibility endpoint is unsuitable for this tool-calling profile.

Model or image does not work. Check the exact quantized model tag, `input: ["text", "image"]`, and the image-model selection. Send a PNG/JPEG screenshot under 5 MB with one crop at a time. Enlarging the crop's text may help more than sending a larger full-screen image. A small model can still misread it.

Timeout or very slow first reply. CPU speed is hardware-dependent. The provider timeout is 600 seconds and the agent turn budget is 900 seconds. Check CPU saturation, competing processes and model loading before increasing them. Make tasks and file reads smaller; longer timeouts do not improve throughput.

OOM, restarts or constant swapping. Stop competing applications; verify the exact 2B Q4 model and one-runner settings. Reduce the image longest side from 1024 to 768 and retry the full acceptance check. Do not increase caps beyond the host budget. If this profile still fails, record it as unsupported on that host; do not claim that the download size proves fit.

Permission denied on user files. Regenerate local UID/GID as the owning normal Linux user. Check ownership of `.state/` and `workspace/`; repair only these project directories if a previous root-run setup created them. Prompt files are intentionally read-only inside the container.

Docker Desktop: mountpoint is outside of rootfs. Update with `git pull --ff-only`, regenerate config, and recreate OpenClaw. The current configuration uses directory mounts to avoid the [VirtioFS nested file-bind failure](#). Run the container mount check in README before repeating the plugin installation if it failed earlier. Keep the existing model volume. For pending plugin migration warnings, see README's diagnostic command.

10. Validation record and references

What was checked for this release

Static checks use synthetic Discord IDs and tokens, never a real account. Generator tests cover multiple users, exact channel routing, rejection of wildcard/invalid IDs, and keeping the token out of JSON. Compose validates without starting a daemon. The OpenClaw 2026.9.8 CLI validates the generated configuration with the pinned official Discord plugin installed in temporary state.

Image manifests were checked for Linux amd64 availability. The supplied PDF was rendered and visually reviewed. See docs/validation.md for the final commands and results, and docs/versions.json for recorded image identities.

Verification	Status
Ubuntu tools, Docker install, hello-world	Instructions checked; not run on Linux
Generator tests, Compose, OpenClaw schema	Checked locally; see validation record
Official image manifests, Linux amd64	Checked; image tags recorded
Container mounts and prompt protection	Mounts checked; inference not run
Discord image/file delivery and access checks	Requires your bot and target host
Whole-host 8 GB RAM / CPU performance	Not measured; use page 8

Do not treat a raw model smoke test as proof of an agent turn: the latter adds system instructions, tools, history, images and possibly multiple model requests. Keep the prompt short, test representative tasks, and preserve the distinction between configuration validation and a live result.

Primary sources

The guide targets the pinned releases below. Upstream web documentation can evolve; the tagged source documentation is the compatibility reference for this configuration.

- [OpenClaw 2026.9.8 release](#)
- [Tagged Docker setup and image behavior](#)
- [Tagged Ollama configuration recipes](#)
- [Tagged Ollama context, thinking and runtime options](#)
- [Tagged system-prompt composition](#)
- [Tagged Discord setup](#)
- [Tagged Discord access control](#)
- [Qwen3.5 2B Q4_K_M model and vision projector](#)
- [Ollama Docker instructions](#)
- [Ollama 0.40.0 release](#)
- [Docker Compose service resource limits](#)

Rebuild the PDF

```
python3 -m venv .venv
.venv/bin/pip install -r requirements-pdf.txt
.venv/bin/python scripts/build_pdf.py
```